SceneMinds

SchubKI National AI Hackathon 2026

LogBase.

powered by

SereneDB

The Challenge & the Data.

SceneMinds

Same logs. Three roles.

Millions of log lines won't fit in one prompt, and an LLM on its own can invent numbers.

Engineer

What happened around this failure?
Inspect exact records, source identities and nearby events. Separate observed facts from possible explanations.

Team lead

Which problems keep coming back?
Compare counts, trends and components. Identify recurring patterns and coordinate the next investigation.

Manager

What should we investigate first?
Review ranked log volumes and mapped owners. Prioritize with evidence while acknowledging uncertainty.

BGL 4.75M logs

- 4,747,963 physical log records from a supercomputer. Coverage: June 2005 to January 2006.
- 4,747,953 parsed records; 10 malformed headers retained and flagged instead of silently discarded.
- Node, label and log type are preserved alongside the shared timestamp, level and component fields.

OpenStack 207,820

- 207,820 physical log records from three cloud log files. Coverage: May 14–17, 2017.
- Request IDs, instance IDs and correlation context support investigations where those fields are available.
- Both datasets retain original text and source-file/line identities. Different years are never one incident timeline.

The Solution?

LogBase.

SereneDB searches and counts. The agent explains. Every number comes from a query that ran.

Question → Evidence → Recommendation

How LogBase works

01 User

Asks a question and provides their role.

Receives findings, charts and next steps.

↔

02 AI agent

Chooses tools and filters.

Interprets evidence; follows up when needed.

↕

04 SereneDB + data

Searches BGL + OpenStack logs and ownership.

Calculates counts, trends and term statistics.

↔

03 LogBase tools / MCP

Translate tool calls into scoped SQL.

Return results, log references and queries.

Request: 1 → 2 → 3 → 4 | Evidence and answer: 4 → 3 → 2 → 1

A modular architecture

01 MCP client

An existing AI assistant handles the conversation and reasoning. Client capabilities determine how charts are rendered.

02 FastMCP server

Exposes documented, read-only tools with typed inputs. Shared instructions guide investigation, role-specific answers and evidence use.

03 Query layer

Tool modules build scoped SQL queries through one shared connection layer. Local Docker and direct connection setups are documented.

04 SereneDB

Stores and searches the loaded logs, joins ownership mappings and computes aggregates. The model explains database results.

The Tools.

Eight investigation tools to:

01 Discover scope

data_overview

Returns dataset size, coverage and valid filters so the assistant starts with the available evidence.

02 Inspect evidence

search_logs • nearby_events • get_event_details

Find matching records, inspect relevant context and retrieve complete source evidence.

03 Measure patterns

count_by • trend • top_problems

Calculate grouped counts, changes over time and ranked problem volumes with mapped owners.

04 Find distinctive terms

distinctive_terms

Compare a focus period with a baseline to surface unusually common terms, with support counts and explicit scope.

Phrase Matching

- Real search syntax — phrases, AND, OR, and exclusions — over SereneDB's inverted full-text index
- keeping the word "error" distinct from the log severity ERROR.

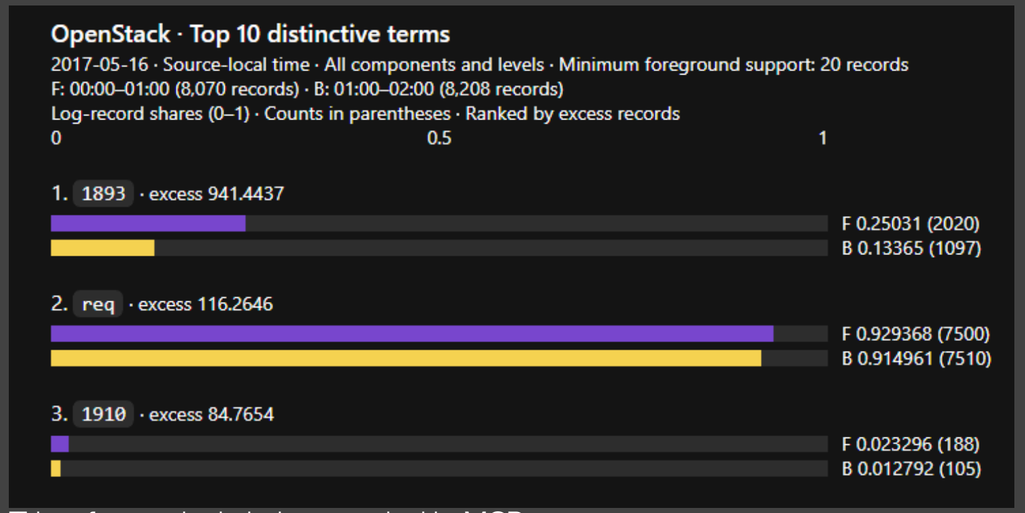
```sql
-- Count every match (unlimited)
SELECT COUNT(*) AS matches
FROM openstack_logs_fts
WHERE content @@ websearch_to_tsquery('"disk info" -debug')
  AND level IN ('ERROR')
  AND event_time >= '2017-05-16 00:00:00'
  AND event_time <  '2017-05-17 00:00:00';

-- Return a bounded, ranked set of evidence
SELECT source_file, line_id, event_time, level, component,
       LEFT(content, 500) AS content_preview
FROM openstack_logs_fts
WHERE content @@ websearch_to_tsquery('"disk info" -debug')
  AND level IN ('ERROR')
ORDER BY event_time, source_file, line_id
LIMIT 10;
```

- Query parsing runs in SereneDB via websearch_to_tsquery:
- "disk info" → exact phrase
- space → implicit AND
- -debug → exclude
- OR → alternatives

Distinctive Terms

- Distinctive Terms (one-liner): Finds terms over-represented in a period vs. a baseline using SereneDB's native ts_dict_agg/ts_dict_count term aggregates
- all counting happens in the database, never in Python.



Taken from actual chat connected to MCP

For more info on the other tools created, check out the GitHub Repo

The Agent.

What the tools return

- KERNEL and APP account for 84.5% of matching problem-level log records in the demonstrated full-dataset ranking.
- KERNEL: 626,756 records (61.92%). APP: 228,536 records (22.58%). Denominator: 1,012,140 matching records.
- Source: LogBase top_problems demonstration. BGL, June 3, 2005–January 4, 2006; configured problem levels.

What the Agent says

- Manager: begin the next investigation with these components. The mapping points to Compute Reliability and Application Operations.
- Team lead: compare recurrence and time windows. Engineer: inspect representative full records and related events.
- This is log volume, not a count of unique incidents or measured business impact. Team ownership is fictional demonstration data.

SceneMinds

Grounded insights

EXACT RECORDS

Full records retain their raw text and original fields. Dataset, source file and line ID identify the evidence. Nearby events are scoped to relevant context where available; adjacent lines alone do not establish a connection.

FAIR COMPARISONS

Counts include the time range, filters and denominator. A rise in error volume is checked against overall log volume where appropriate. Log frequency is a proxy for activity, not a measured request or failure rate.

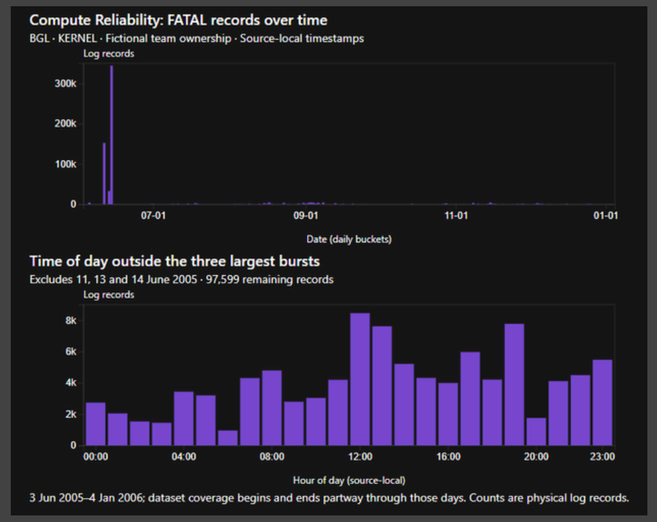
UNCERTAINTY

Separate observations, hypotheses and next checks. Temporal order and distinctive terms suggest leads, not proven causes. When evidence is missing, say so. Fictional ownership remains clearly labelled.

Extra Features.

Visual answers

- Data Visualisation, only when it aids the investigation
- Charts are requested by default for useful numerical comparisons, rather than only when the user explicitly asks for a graph.
- The connected client renders the visualization. Presentation instructions guide the result.



Investigation Recap

- After a meaningful investigation, the assistant offers a recap. With user agreement, it explains the question, checks, findings and next steps as a concise narrative.
- Cited events use full raw logs and labelled fields, including source identities. Evidence is retrieved from tool results rather than reconstructed from memory.
- Relevant charts accompany the findings, with the same scope, filters and denominators. Rendering depends on the connected client.
- An appendix groups the tools used and their executed SQL. Distinct queries remain visible when one tool was called multiple times.

SceneMinds

Limitations

NO FEEDBACK LOOP

We work with static logs and no access to the live systems behind them, so the agent can suggest a fix but we cannot apply it and re-read the logs to confirm it worked. A practical evaluation loop would have measured how good the insights really are.

SESSION-SCOPED RECAPS

Investigation recaps are built from retained conversation context, not a durable store; if history is compacted or the session restarts, the agent discloses the gap rather than reconstructing it.

GUIDED; NOT GUARANTEED

The server guides the agent (auto-charting, role-aware answers, recaps), reasoning and presentation could therefore vary between different LLMs

Checkout LogBase

- Repository and setup guide:
- github.com/SceneMindsHack/LogBase
- Demonstration:

LogBase Demo

Next steps

- Evaluate: Build a repeatable set of strong, ambiguous and missing-evidence cases; compare claims with database results.
- Refine: Improve consistency of charts and recaps across clients.
- Scale: explore foreign-data indexing and object storage.